

Chapter 3. Actors and Scaling Large Systems

One actor is no actor. Actors come in systems.

—Carl Hewitt

As discussed previously, actors can create other actors in a supervisor-worker relationship, where the supervisor delegates tasks to the workers. In the previous example, the supervisor delegated specific subtasks to the workers. Another common pattern provides a level of elasticity as more work requests arrive, and then the supervisor delegates the tasks to idle workers. If there are no idle workers, then the supervisor adds more workers (see [Figure 3-1](#)).

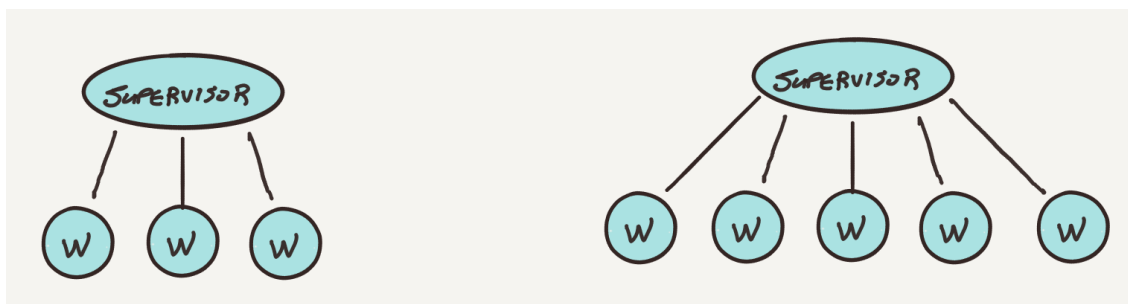


Figure 3-1. Supervisor adds more worker actors as the processing load increases

How the supervisor decides to add workers is up to the supervisor. Supervisors typically have some limit as to how many workers they will add, as well as a built-in way for shedding excess idle workers (see [Figure 3-2](#)).

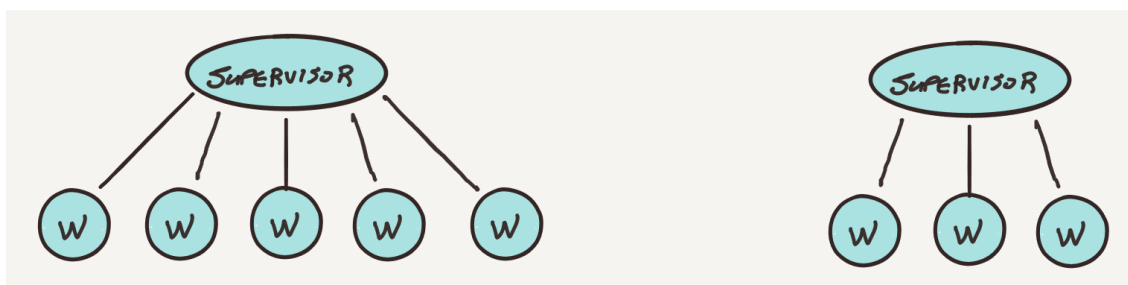


Figure 3-2. The supervisor sheds workers as the load decreases

Here again the scheduled timeout mechanism comes into play. Each of the workers could schedule an idle timeout message to be sent at some point in the future. Every time a worker gets a request to perform a task,

it resets the timeout. If no tasks are sent to a worker, and it receives the timeout message that tells the worker that it has been idle for too long, it triggers a shutdown of itself.

Using this relatively simple pattern provides for elastic scalability on the actor level, where a supervisor is free to expand out when the workload increases and contract in when the workload decreases. Other strategies and technologies may be used to support various forms of elasticity across the complete system, but the basic functionality of actors allows for a lot of creativity in how application systems may be designed and implemented.

Another point to be made here is that the supervisor is also an actor. From the perspective of other actors that send it messages, the fact that the supervisor is delegating tasks out to workers is completely transparent to the senders. Imagine again sending a friend a text message asking him to perform some task. You have no idea how he may actually carry out that task. Your only concern is that the task gets completed, and it gets completed in the time frame that you expect. If it happens that he is delegating these tasks to a team of workers, it is none of your concern. It is also transparent to you that while he's working on your task, he may also be concurrently working on tasks for others.

A Look at the Broader Actor System

Actors exist within an *actor system*, and the process of actors sending asynchronous messages to each other is handled by the actor system. Yet an actor system provides much more than just the protocol for messaging between actors; it is also designed to manage actor activities by allocating the system resources needed to support the actor environment.

The following section covers how the actor system coordinates the activities of the actors and how the actor system allocates system resources at a conceptual level. We use an analogy of comparing actors working in the actor system to that of office workers working in office spaces. The goal here is to provide some insight and intuition of the mechanics of how the actor system runs without going too deeply into the technical details.

Virtually every computer system is composed of a set of processing cores, some memory (RAM), and other resources such as disk space, network interfaces, and other devices. In this discussion the focus is on memory and processing cores.

When programs run in a computer system, the actual work is handled by what is called a *thread* or *thread of execution*. Just as each computer system has a limited number of cores and a limited amount of memory, there is also a limited number of threads. A typical computer system has a small number of cores, and common small or medium-sized servers may have 4 to 16 processing cores. (Just looking at [Amazon Web Services](#) instances, you can see it's possible to provision cloud instances with 128 processor cores and 2 terabytes of in-memory RAM.) For memory (RAM), this varies from a few gigabytes to terabytes. A typical server has 8GB to 64GB, but this number keeps going up as the cost of memory decreases in light of technological improvements. For threads, the count typically falls into the hundreds range.

The number of cores and the amount of memory dictates how much work a given computer system is capable of handling. This is roughly analogous to a room. Depending on the size of an office space, more workers will fit in the room if it is larger, which means it can handle more work (threads).

How Actors Manage Requests

In this example scenario, we have a room that can fit three desks at which workers sit and do work. Requests to do work come into the office, with each task delegated to an available worker. Workers can only work on their assigned tasks when the desk has power, which in this example scenario is a limited resource (see [Figure 3-3](#)).

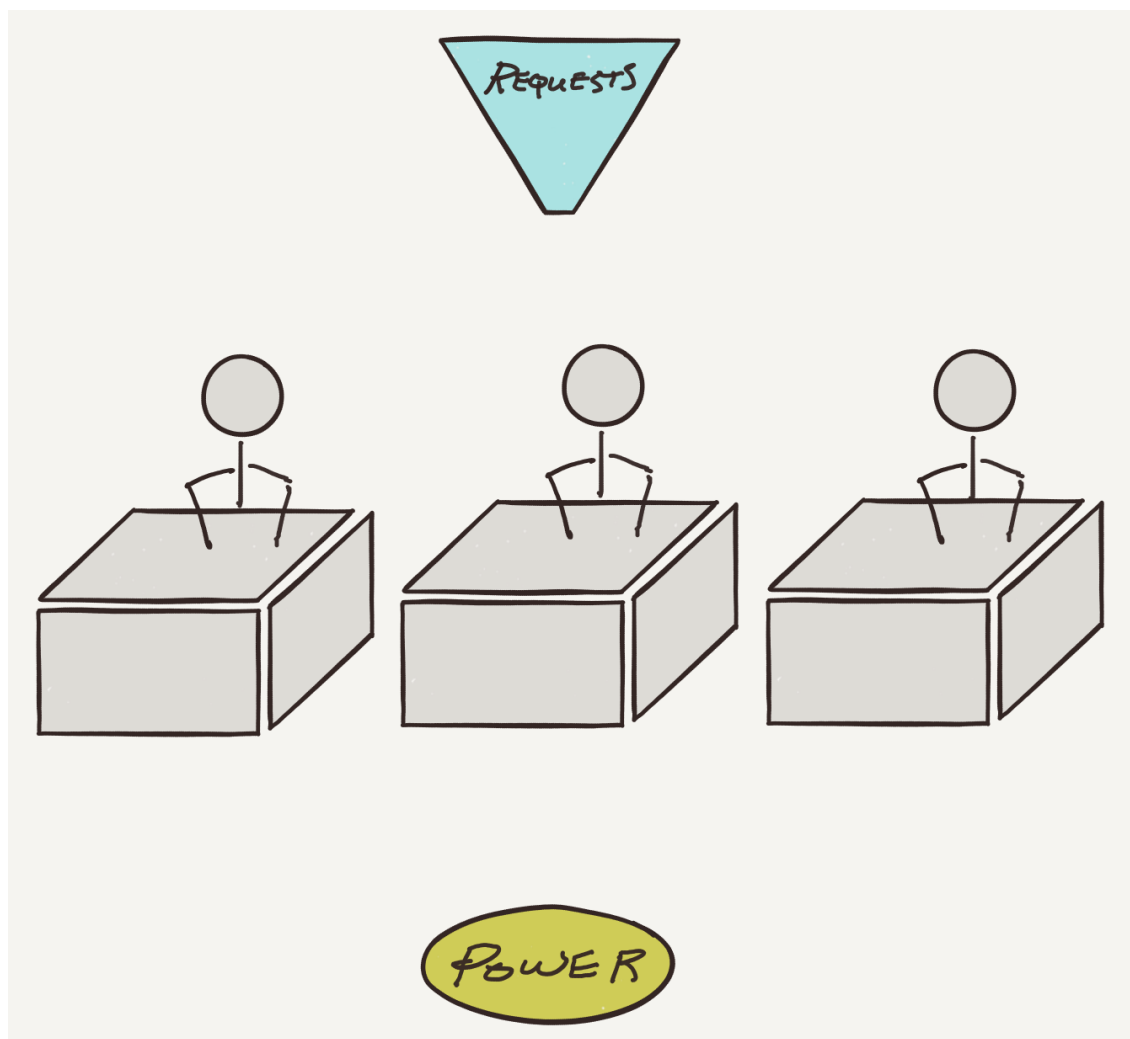


Figure 3-3. An office example where workers sit at desks to perform tasks only when the desk has power

We are going to use this office space analogy to discuss how the actor system coordinates actor activity in a computer system. The idea is that the room size determines how many desks will fit into a room. This is an attempt to model how the memory size of a given computer system determines how many threads can be supported. So room size is analogous to memory size, and the number of desks is analogous to the number of threads. Workers sit at desks to get work done.

To model the processing cores, the idea is that a worker sitting at a desk can only do work when the power is turned on (see [Figure 3-4](#)). This demonstrates how a given thread is only active when it is given a processing core to run on. Compared to the number of available cores on which to do work, there are many more threads representing the requests for work to be done. In this office analogy, there are a limited number of desks that can be powered on at any point in time.

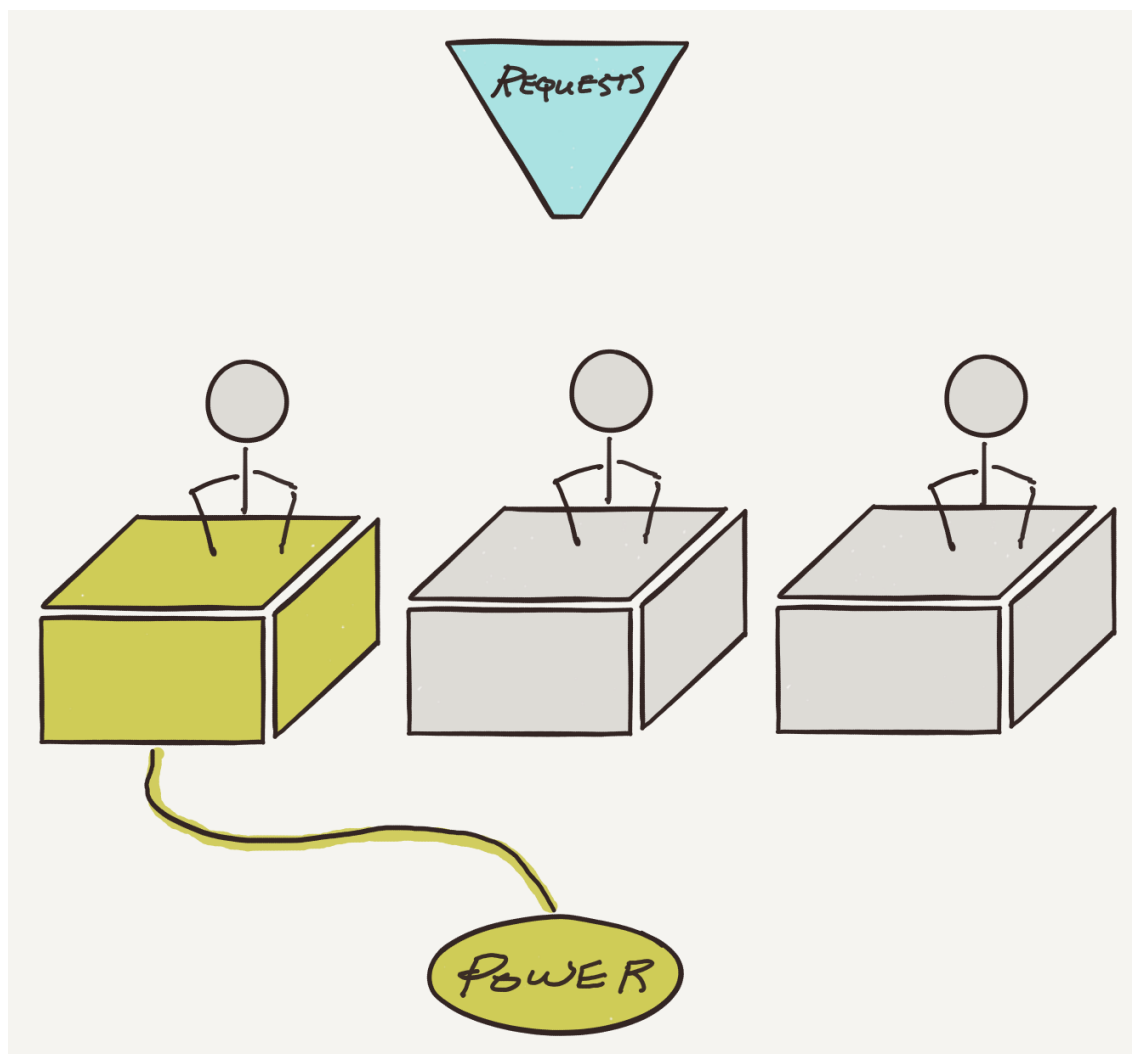


Figure 3-4. One desk has power, so the worker at that desk may work on its assigned task

Now imagine that in this hypothetical office the workers perform tasks that are coming in from the outside. For example, customers are sending in text messages to the office. These text messages are delegated to available workers sitting at the desks (see [Figure 3-5](#)).

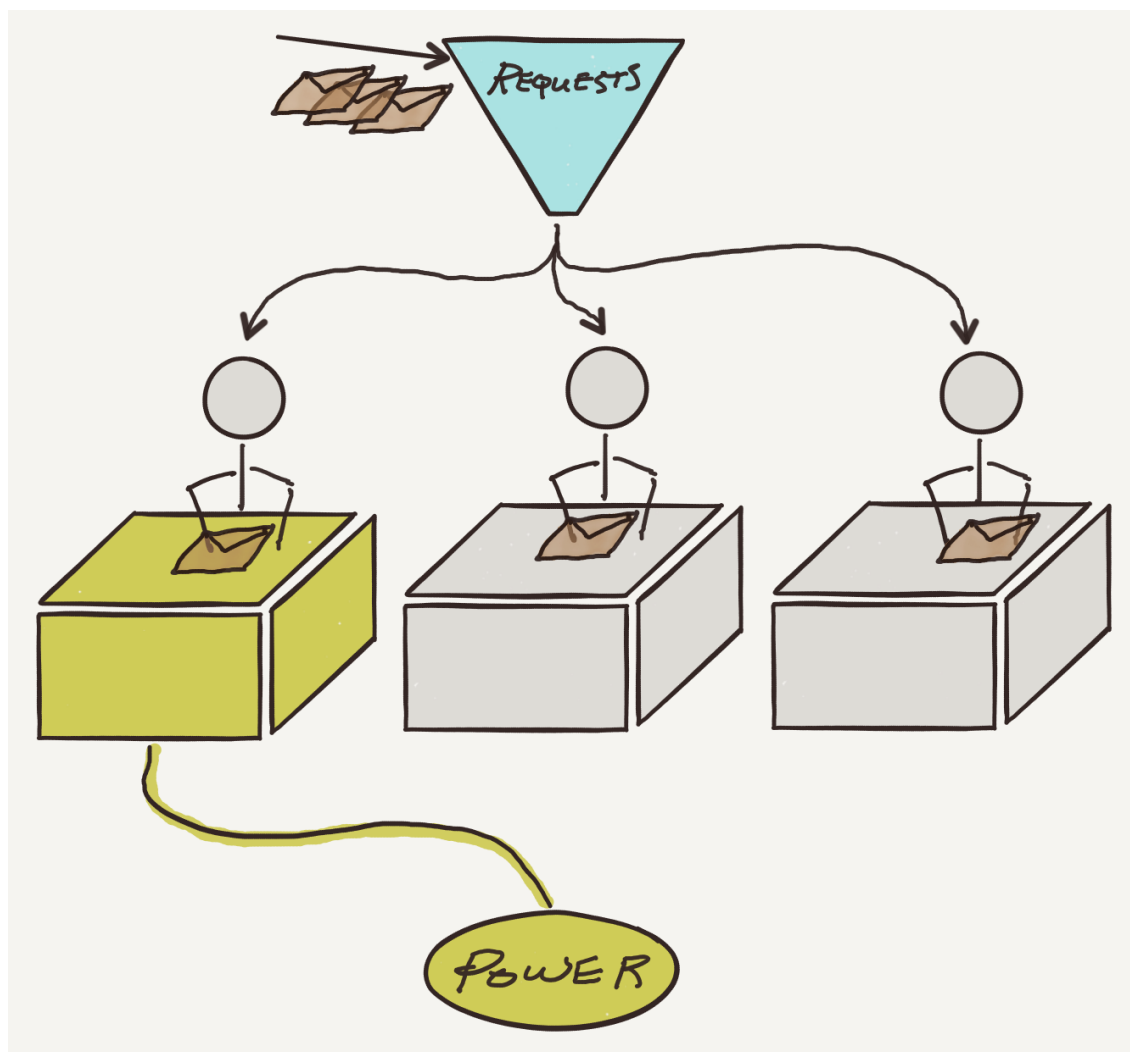


Figure 3-5. Messages are sent to the office and then routed to each worker

The power is constantly being shifted between desks to the workers that are ready to process these text messages. It is the responsibility of the system to make sure that no single worker can hog one of the limited power connections (see [Figure 3-6](#)).

This works the same in a computer system, which must constantly shift from thread to thread, allocating limited processing cores while trying to maintain a fair utilization of cores with all of the threads that are ready to run. So in the same way that our office example has more desks than available power connections, computer systems also have many more threads than available processing cores.

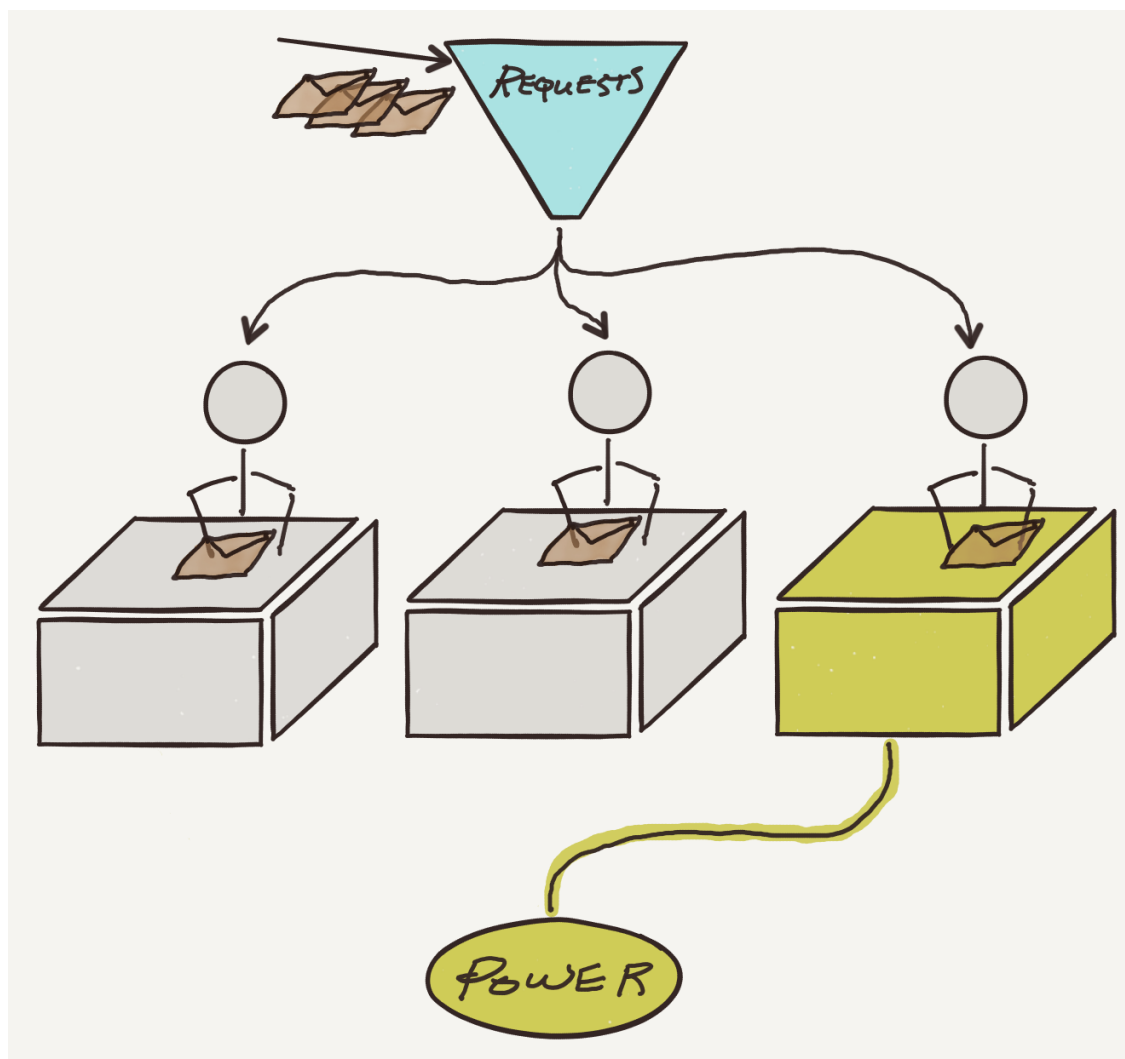


Figure 3-6. The system swaps the power between desks to allow an even distribution of work

Traditional Systems Versus Actor-based Systems

So how does this work in traditional, synchronous systems versus those built on the Actor model?

This constant shifting of the processing cores from thread to thread happens in all computer systems. When running traditional synchronous systems versus actor-based asynchronous systems, the difference is how the threads themselves are managed.

With synchronous systems, a thread is assigned to a request and it handles it until the processing of the request is completed with a response. This also includes holding the thread while waiting for I/O to complete. With asynchronous systems, threads are managed more dynamically—threads are only allocated to actors when they are ready to run and they are ready to use an available processing core. When an actor must stop

and wait for an I/O to complete, *the thread is released* and made available to be allocated to another actor.

In a synchronous software system, the common approach is that when a request is received it is delegated to a thread and that thread is responsible for handling that request until it is completed and a response is returned. In our office analogy, this is modeled with text messages sent to the office, where the office delegates the text message to a worker sitting at a desk, and that worker handles that request until it is completed. Once the office worker is assigned to a desk, she will stay at that desk until the given task is completed.

With the synchronous processing approach, the system limit for how many requests can be processed concurrently is limited by the number of threads. With the text messages example, the limit of the concurrently handled text messages is the number of desks. If the office space allows for 100 desks then that office can concurrently handle 100 text messages. When all of the workers at the desks are busy handling text messages, if more text messages arrive they will not be processed until some of the workers complete their current task and are ready to do more work. When text messages arrive faster than they can be processed the system needs to either put the pending requests into a queue or reject the excess messages (see [Figure 3-7](#)).

In addition, with the synchronous processing approach, when the processing must be suspended while waiting for various I/O operations to complete, the thread that is handling the request is idle, *yet still in use and consuming memory*. This is like the office worker sitting at a desk doing nothing while he waits for some operation to complete.

The idea is that the office worker received the incoming text message, which triggers the office worker to send another text message to someone else to perform some subtask, and the office worker is sitting there idle waiting for a response text message. The result is that the limited threads are in use but not doing any useful work, much like an office worker sitting at a desk doing no useful work while waiting for someone else to finish a subtask. In the office example, the number of desks limits how many text messages can be handled at one time.

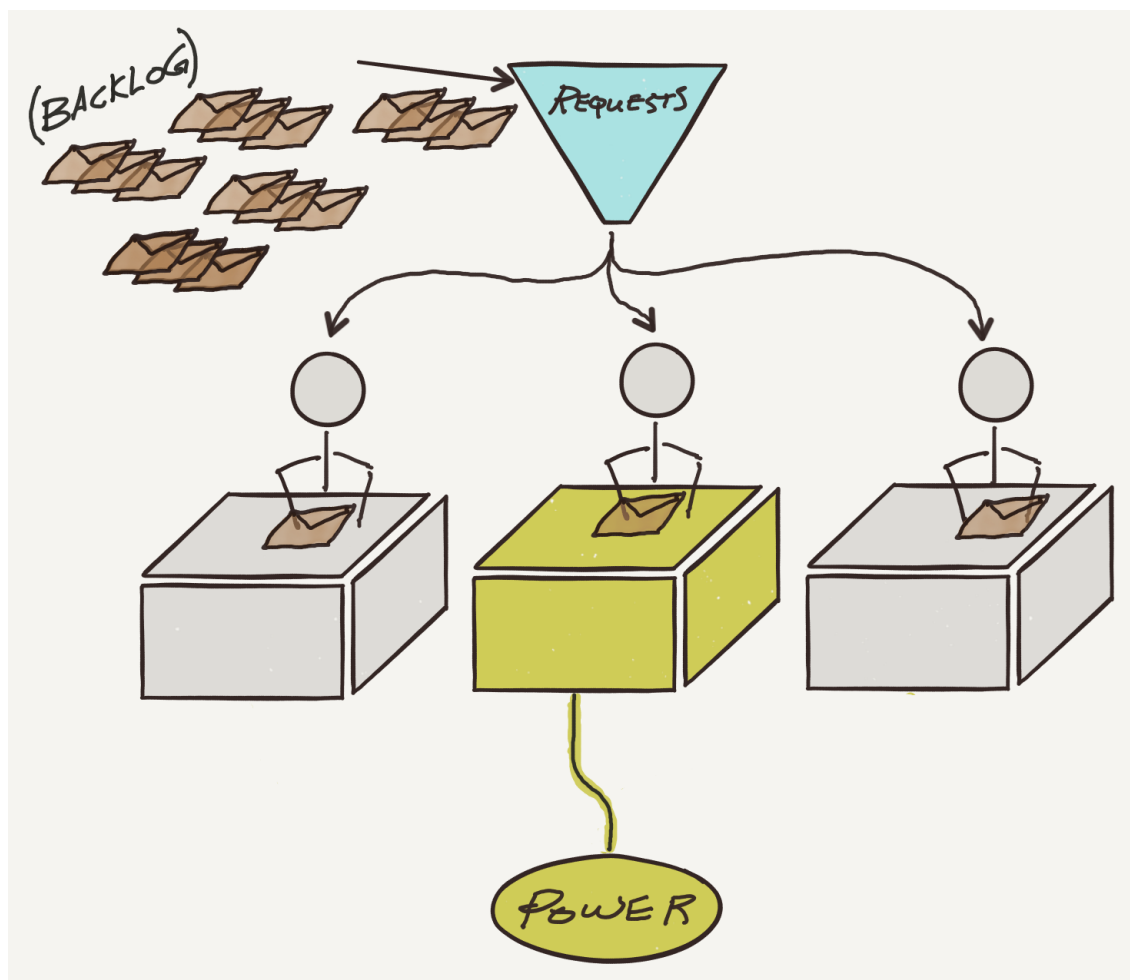


Figure 3-7. When messages arrive faster than the workers can perform each task, then there is a backlog of pending messages that are either queued or discarded

With the Actor model, the dynamics of the system are very different. The actor system processes things asynchronously.

In our office model, workers only occupy a desk when they have something to do. When a worker is idle, she goes to the side of the room and the desk is freed up for other workers to use. In an actor system, threads are allocated to actors that have messages to process. When the actor has no messages to process, the thread is allocated to other actors that have messages to process and that have something to do, so that they are not sitting idle while waiting for something like an I/O to complete (see [Figure 3-8](#)).

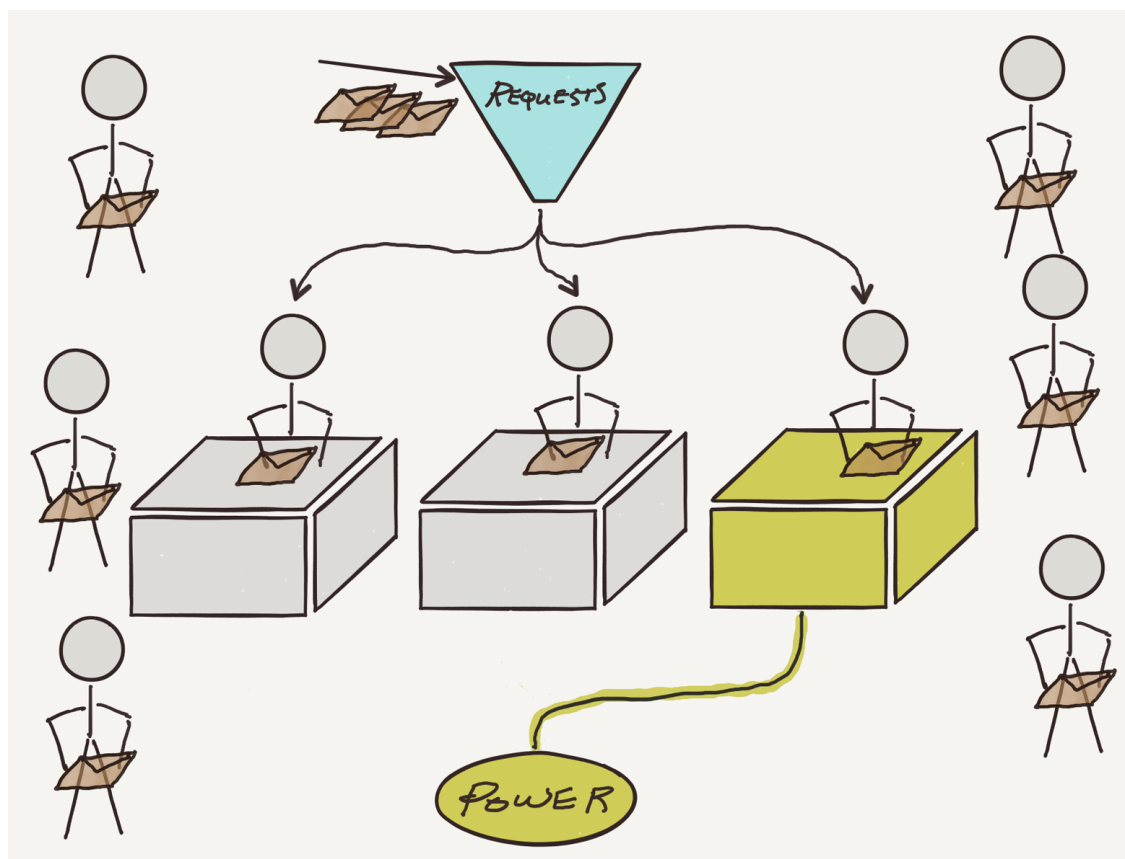


Figure 3-8. In an asynchronous system, workers only occupy desks when they have something to do

At the same time, no single actor can squat on a thread doing nothing. Even when an actor still has more messages to process, it is given a limited amount of time with a thread. The system is constantly moving the threads around to various actors trying to maintain a fair distribution of thread usage without letting some actors dominate their time with threads or, conversely, let some actors be starved for attention.

The end result is that asynchronous actor systems can handle many more concurrent requests with the same amount of resources *since the limited number of threads never sit idle* while waiting for I/O operations to complete.

This is analogous to the office model. In the synchronous processing flow, there is the possibility for many workers sitting at the limited number of desks to be idle at any point in time. With the asynchronous processing flow, a worker that is idle or has nothing to do goes to the side of the room, which frees up the desk for another worker that is ready to do some work.

Expanding into Clusters of Actors

The story does not end there. Say things get to the point where the rate of work to be done has outgrown the maximum message processing rate of the current office. Even with the use of asynchronous actors and their efficient use of a single system, the level of activity may still get to the point that the flow of work to be done exceeds the capacity of the system to process it. To address this the actor system can form clusters. A cluster is where two or more systems, each running an actor system, are configured to work as a collaborative group (see [Figure 3-9](#)).

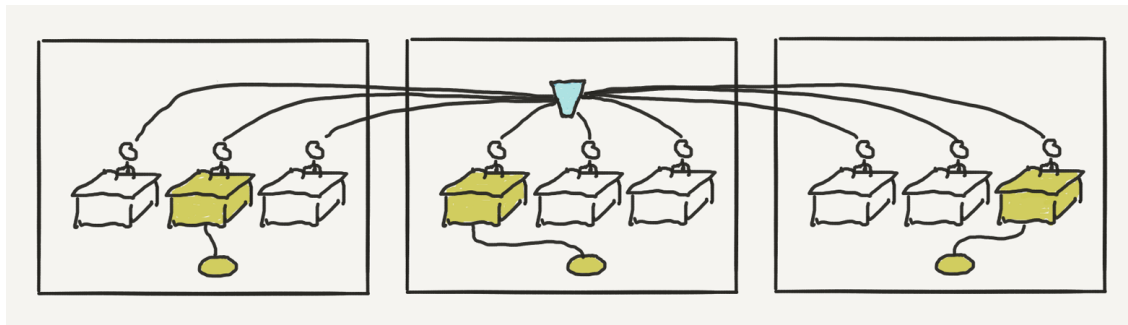


Figure 3-9. Actor systems may run in a cluster

From the perspective of the actors running in a cluster, there is no difference between sending messages to actors in the same system, or between actors on different systems because the actor system still handles all the actual messaging between actors. Of course, messaging between actors on different systems takes more time (i.e., in the milliseconds range) than messaging on the same system. However, the actor system is optimized to handle both the local and remote forms of messaging. The typical actor system is capable of handling millions of messages per second, and this applies from a single system cluster to many system clusters.

The actor system cluster capability provides the building blocks for elastically expanding the processing capacity as the load goes up and contracting the capacity as the load goes down. This level of elasticity cannot be patched on after the fact, but must be designed as a part of the architecture of each individual application implementation.

In some implementations, there are manual processes used to expand and contract the cluster as needed. In other implementations, elasticity is handled automatically by the application. The key point here is that the actor system provides all the building blocks needed for creating clusters, which can then be used to architect, design, and implement distributed systems that can elastically expand and contract as needed.

This is in stark contrast to the traditional ways we have been building systems for the past decade and longer. With older system architectures, we are typically forced to scale vertically, using larger and larger systems and an ever-expanding infrastructure footprint. In these systems, there were limited options to distribute the work horizontally across multiple systems. In addition, the scale of these traditional synchronous systems is static, in that these systems were sized to reasonably handle peak loads. This results in low resource utilization because during slow times you are paying for wasted capacity that is sitting idle and, to make matters worse, it is extremely difficult to predict what the peak loads will be. The typical results are when there are peak periods of activity and capacity is insufficient to handle the load. When this happens, the system users suffer annoying response times. Annoyed users are more often former users because in many cases they can take their business elsewhere in an instant.

Ultimately, asynchronous processing provides a way of doing more work with less processing capacity. Clustering provides the building blocks for building application systems that grow and contract as the processing load requires. These two features of the actor system directly impact the operational costs of your application system: you use the processing capacity that you have more efficiently and you use only the capacity that is needed at a given point in time.

The main takeaways in this chapter are:

- Delegation of work through supervised workers allows for higher levels of concurrency and fault tolerance.
- Workers are asynchronous and run concurrently, never sitting idle as in synchronous systems.
- Efficient utilization of system resources (CPU, memory, and threads) results in reduced infrastructure costs.
- It's simple to scale elastically at the actor level by increasing or decreasing workers as needed.
- Using clusters gives the ability to scale at the system level.

